REPOPILOT

API Documentation

Version 1.0 · Proposed implementation contract · 31 October 2026

From GitHub issue to verified, review-ready pull request.

This document defines the HTTP API for RepoPilot: a repository-aware AI engineering agent that connects to public GitHub repositories, investigates issues, proposes code patches, runs tests in an isolated Docker environment, produces an evidence report and prepares a pull request after explicit human approval.

Document status: This is a detailed API design aligned to RepoPilot PRD v1.0. The PRD defines endpoint families and product requirements; request/response fields, status codes and several operational details below are proposed contracts for implementation and should be treated as the v1 API specification.

Document property	Value
Product	RepoPilot
API version	v1
Base path	/api
Transport	HTTPS + JSON; Server-Sent Events for run events
Primary integrations	GitHub API, Gemma 4 inference, Docker worker
Audience	Frontend developers, backend developers, QA and integration engineers

Confidentiality: Engineering design document. Do not place GitHub credentials, model API keys or secrets in client-side code.

Contents

• 1. Overview and design principles
• 2. Environments, base URL and conventions
• 3. Authentication and authorization
• 4. Common request and response format
• 5. API endpoint summary
• 6. Authentication endpoints
• 7. Repository endpoints
• 8. Issue and analysis data models
• 9. Agent execution endpoints
• 10. Patch review and test execution
• 11. Approval and pull-request publishing
• 12. Patch Evidence Report
• 13. Server-Sent Events contract
• 14. Data models and state machine
• 15. Error catalogue
• 16. Security and operational requirements
• 17. Idempotency, pagination and rate limits
• 18. Environment variables
• 19. End-to-end integration example
• 20. Acceptance and test checklist

1. Overview and design principles

RepoPilot exposes a versioned REST API under /api. The React client calls the Express/TypeScript backend. The backend owns GitHub credentials, validates all requests, coordinates Gemma 4, retrieves bounded repository context, validates patches, delegates test execution to a restricted Docker worker and stores execution evidence in MongoDB.

Principle	API consequence
Human approval first	Creating a pull request is a separate, explicit action after review; no automatic merge.
Evidence over claims	Test status is based on actual command output, exit code and duration. Untested changes are labelled.
Backend-controlled tools	The model can suggest an action; the backend validates and executes only allowlisted operations.
Bounded context	Repository files and issue text are retrieved selectively and size-limited.
Public repository MVP	Only public GitHub repositories are in scope for v1.
One orchestrator	Agent stages are coordinated by one backend workflow, not separate autonomous agents.

1.1 High-level request flow

Browser → Express API → authentication/authorization → GitHub service and repository retrieval → agent orchestrator → Gemma 4 service → patch validator → Docker worker → evidence report → user approval → GitHub fork/branch/commit/PR.

1.2 Scope boundaries

Included in v1: GitHub OAuth, public repository connection, open issue discovery, issue classification, repository investigation, root-cause hypotheses, patch generation, patch validation, Node.js Docker test execution, event streaming, evidence reports and approval-gated PR creation. Private repositories, automatic merging, enterprise team management, arbitrary language runners and browser-based IDE functionality are out of scope.

Important: A successful test run means the selected command passed in the configured sandbox. It does not prove that the patch is defect-free or that every possible test passed.

2. Environments, base URL and conventions

Environment	Base URL example
Local	http://localhost:8001/api
Production	https://api./api

All production traffic must use HTTPS. The examples use JSON request/response bodies unless an endpoint is explicitly marked as an SSE stream or multipart upload. API versioning is in the path; this document defines v1 under /api.

2.1 Common headers

```
Content-Type: application/json
Accept: application/json
Authorization: Bearer <application_session_token>
X-Request-Id: <client-generated-uuid>
Idempotency-Key: <unique-key>   # required for PR creation; recommended for run creation
```

The proposed browser implementation may use secure, HttpOnly, SameSite cookies instead of a bearer token. Choose one session strategy consistently; never expose the GitHub OAuth access token to the browser.

2.2 Identifiers and timestamps

Application resource IDs are opaque strings (MongoDB ObjectId serialized as a string is acceptable). GitHub repository and issue numbers are numeric. Timestamps are ISO 8601 UTC strings, for example 2026-10-31T09:30:00.000Z. Durations are milliseconds.

3. Authentication and authorization

Authentication uses GitHub OAuth. The backend exchanges the OAuth code, stores GitHub credentials securely server-side and creates an application session. The frontend receives only the application session or secure cookie. OAuth state must be random, short-lived and validated to prevent CSRF.

3.1 Permission model

Capability	Required access
Read public repositories and issues	Public read access; no write permission required.
Fork repository	Authenticated GitHub identity and permission to create a fork.
Create branch/commit in user's fork	Write access to the user's fork.
Open pull request against upstream	GitHub permission to create a PR from the fork; upstream repository must accept contributions.
Merge pull request	Not supported by RepoPilot v1.

3.2 Authorization rules

Every protected request must resolve the authenticated user from the server-side session. A user may access only their own agent runs, reports and connected-repository records. Do not trust userId supplied in a request body. Verify ownership on every runId/repositoryId lookup. GitHub OAuth tokens must never be logged, returned in API payloads or sent to Gemma 4.

4. Common request and response format

4.1 Successful response envelope

```
{
  "success": true,
  "data": { },
  "meta": {
    "requestId": "req_01J...",
    "timestamp": "2026-10-31T09:30:00.000Z"
  }
}
```

4.2 Error response envelope

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request contains invalid fields.",
    "details": [
      { "field": "repositoryUrl", "message": "Must be a public GitHub URL." }
    ],
    "retryable": false
  },
  "meta": {
    "requestId": "req_01J...",
    "timestamp": "2026-10-31T09:30:00.000Z"
  }
}
```

Never return stack traces, internal file paths, secrets, raw provider credentials or container host details to clients. Detailed diagnostics belong in server logs keyed by requestId.

5. API endpoint summary

Method	Endpoint	Purpose	Auth
POST	/auth/github	Exchange OAuth code / establish session	OAuth flow
GET	/auth/me	Current user profile	Required
POST	/auth/logout	End application session	Required
POST	/repositories/connect	Connect a public GitHub repository	Required
GET	/repositories/:id	Repository details	Required
GET	/repositories/:id/issues	List open issues	Required
GET	/repositories/:id/issues/:number	Issue details	Required
POST	/agent/analyze	Start an agent run	Required
GET	/agent/:runId	Get run status and results	Required
GET	/agent/:runId/events	Stream run events (SSE)	Required
POST	/agent/:runId/test	Start/retry sandbox tests	Required
POST	/agent/:runId/approve	Approve publishing	Required
POST	/agent/:runId/pull-request	Create pull request	Required
GET	/reports/:runId	Retrieve evidence report	Required

The routes above consolidate the PRD endpoint list and add supporting read endpoints for a complete frontend workflow. The additional routes are proposed implementation details.

6. Authentication endpoints

6.1 POST /auth/github

Purpose: complete the GitHub OAuth callback exchange. In a standard browser OAuth flow, GitHub redirects to a backend callback route with code and state. If the frontend posts the code, the backend must still validate state and redirect origin.

Request body

```
{
  "code": "github_oauth_authorization_code",
  "state": "random_state_value"
}
```

Response — 200 OK

```
{
  "success": true,
  "data": {
    "user": {
      "id": "usr_...",
      "githubId": 12345678,
      "username": "octocat",
      "avatarUrl": "https://avatars.githubusercontent.com/u/...",
      "name": "The Octocat"
    },
    "session": { "expiresAt": "2026-11-07T09:30:00.000Z" }
  },
  "meta": { "requestId": "req_..." }
}
```

The session should normally be set as a Secure, HttpOnly, SameSite cookie. Do not include the GitHub access token in this response.

Possible errors: 400 OAUTH_STATE_INVALID, 401 OAUTH_CODE_INVALID, 502 GITHUB_PROVIDER_ERROR, 429 RATE_LIMITED.

6.2 GET /auth/me

Returns the authenticated application user. Response data contains user id, GitHub id, username, avatar URL and display name. Returns 401 when no valid session exists.

```
{
  "success": true,
  "data": {
    "id": "usr_...",
    "githubId": 12345678,
    "username": "octocat",
    "avatarUrl": "https://avatars.githubusercontent.com/u/...",
    "name": "The Octocat"
  }
}
```

6.3 POST /auth/logout

Invalidates the application session and clears the session cookie. Response: 200 { success: true, data: { loggedOut: true } }. Logout should be safe to retry.

7. Repository endpoints

7.1 POST /repositories/connect

Purpose: validate a GitHub URL and connect a public repository. The backend normalizes owner/name, checks visibility, retrieves metadata, README and a bounded file tree, then stores a repository record.

Request

```
{
  "repositoryUrl": "https://github.com/owner/repository"
}
```

Response — 201 Created

```
{
  "success": true,
  "data": {
    "repository": {
      "id": "repo_...",
      "githubRepositoryId": 123456789,
      "owner": "owner",
      "name": "repository",
      "fullName": "owner/repository",
      "htmlUrl": "https://github.com/owner/repository",
      "defaultBranch": "main",
      "primaryLanguage": "TypeScript",
      "description": "Example repository",
      "visibility": "public",
      "readmeSummary": "A short repository summary.",
      "indexingStatus": "completed",
      "connectedAt": "2026-10-31T09:30:00.000Z"
    }
  }
}
```

Validation: accept canonical github.com repository URLs only; reject malformed URLs, unsupported hosts, private repositories and repository paths that do not identify owner/name. Do not accept arbitrary clone URLs or execute repository scripts during connection.

7.2 GET /repositories/:id

Returns the connected repository record and current indexing metadata. Response includes id, fullName, description, defaultBranch, primaryLanguage, readmeSummary, indexingStatus and lastIndexedAt. Return 404 if absent or not owned by the caller.

7.3 GET /repositories/:id/issues

Lists open GitHub issues, excluding pull requests. Supports pagination.

Query parameters

Parameter	Type	Default	Description
page	integer	1	Page number, minimum 1.
perPage	integer	20	Page size, maximum 100.
state	enum	open	v1 supports open; closed may be added later.
labels	string	—	Optional comma-separated GitHub labels.

Response — 200 OK

```
{
  "success": true,
  "data": {
    "items": [
      {
        "number": 128,
        "title": "Fix authentication timeout",
```

```
        "body": "Refreshing a token sometimes times out...",
        "state": "open",
        "labels": ["bug", "authentication"],
        "createdAt": "2026-10-29T11:00:00Z",
        "updatedAt": "2026-10-30T14:00:00Z",
        "htmlUrl": "https://github.com/owner/repository/issues/128",
        "author": { "login": "contributor", "avatarUrl": "https://..." }
      }
    ],
    "page": 1,
    "perPage": 20,
    "hasNextPage": true
  }
}
```


7.4 GET /repositories/:id/issues/:number

Returns full issue details: number, title, body, labels, state, author, assignees, createdAt, updatedAt, htmlUrl and optional linked pull request metadata. Confirm the GitHub object is an issue, not a pull request.

7.5 Repository retrieval limits

The repository retrieval service should fetch the README, tree and likely package manifests first. Source retrieval is bounded by file count, file size and total model-context budget. Exclude binary files, generated bundles, vendor directories, node_modules, secrets and files above configured limits. Store paths and retrieval metadata so the evidence report can explain which artifacts were inspected.

8. Issue and analysis data models

8.1 Issue classification object


```
{
  "category": "bug",
  "summary": "Refresh requests may fail during Redis retry.",
  "expectedBehavior": "A valid refresh token should be retrieved or a typed error returned.",
  "actualBehavior": "The request may time out during a transient connection issue.",
  "missingInformation": ["Reproduction frequency", "Redis client version"],
  "severity": "high",
  "complexity": "medium",
  "confidence": 0.78
}
```


category enum: bug, feature_request, documentation, enhancement, question, other. severity enum: low, medium, high, critical. complexity enum: low, medium, high. Confidence is a model estimate, not a calibrated probability; label it accordingly in the UI.

8.2 Root-cause investigation object


```
{
  "summary": "A retry path may not be used after a transient Redis failure.",
  "possibleCauses": [
    {
      "description": "The refresh handler returns after the first failed lookup.",
      "supportingEvidence": [
        {
          "filePath": "src/auth/session.ts",
          "lineStart": 140,
          "lineEnd": 147,
          "excerpt": "await redis.get(key)"
        }
      ],
      "confidence": 0.72
    }
  ],
  "candidateFiles": ["src/auth/session.ts", "tests/auth.test.ts"],
  "uncertainties": ["No production trace was supplied."],
  "recommendedNextSteps": ["Inspect Redis retry configuration", "Add a regression test"]
}
```

Evidence references must point to retrieved source content. The model must not invent line numbers, file paths or code excerpts. If exact source locations cannot be established, return a broader reference and mark it as unverified.

9. Agent execution endpoints

The agent workflow is a backend-controlled state machine. A run is created asynchronously and progresses through real stages. The client should not infer completion from elapsed time or animated UI.

9.1 POST /agent/analyze

Starts an agent run for a selected issue. The endpoint validates ownership of the connected repository and confirms the issue exists. It creates a persistent run record and queues or starts the orchestrator.

Request

```
{
  "repositoryId": "repo_...",
  "issueNumber": 128,
  "options": {
    "includeDuplicateCandidates": true,
    "maxFilesToInspect": 20
  }
}
```

Response — 202 Accepted

```
{
  "success": true,
  "data": {
    "runId": "run_...",
    "status": "queued",
    "currentStage": "issue_analysis",
    "createdAt": "2026-10-31T09:30:00.000Z",
    "eventsUrl": "/api/agent/run_.../events",
    "statusUrl": "/api/agent/run_..."
  }
}
```

202 means the run was accepted, not that the AI analysis succeeded. The frontend should subscribe to events and/or poll the status endpoint.

9.2 Agent stages

Stage	Purpose	Typical output
issue_analysis	Classify and summarize the issue.	Classification, missing details, severity and complexity estimate.
repository_investigation	Retrieve README, tree, manifests and relevant source files.	Inspected paths and repository context.
root_cause_analysis	Generate evidence-linked hypotheses.	Possible causes, supporting evidence, uncertainty.
patch_generation	Produce a structured change or unified diff.	Patch, explanation and affected paths.
patch_validation	Validate format, paths and applicability.	Validation result and temporary workspace reference.
test_execution	Run configured tests in Docker.	Command, output, exit code and duration.
pr_preparation	Prepare PR title/body and approval package.	PR draft metadata; no publishing yet.

9.3 GET /agent/:runId

Returns current state and any available stage outputs. The response is a snapshot; use SSE for real-time updates.

```
{
  "success": true,
  "data": {
    "runId": "run_...",
    "repositoryId": "repo_...",
    "issueNumber": 128,
    "status": "running",
    "currentStage": "root_cause_analysis",
    "progress": {
```

```
      "completedStages": 2,
      "totalStages": 7
    },
    "investigation": null,
    "relevantFiles": [
      { "path": "src/auth/session.ts", "reason": "Contains refresh handler" }
    ],
    "patch": null,
    "testResults": null,
    "error": null,
    "startedAt": "2026-10-31T09:31:00Z",
    "completedAt": null
  }
}
```


Possible run statuses: queued, running, awaiting_review, completed, failed, cancelled. Stage status may be pending, running, succeeded, failed or skipped. A run can be completed without a published PR if the user chooses not to publish.

9.4 Cancellation

Cancellation is a recommended extension for long-running runs: POST /agent/:runId/cancel with an empty JSON body. It should stop future stages, request worker termination, preserve completed evidence and mark the run cancelled. If cancellation is not implemented in the first MVP, the UI must not show a cancel control.

10. Patch review and test execution

10.1 Patch representation

The API should persist a patch artifact with a base commit SHA, file paths, unified diff and validation state. Prefer unified diff for display and Git application; keep any structured file-change representation internal if needed.

```
{
  "patch": {
    "format": "unified_diff",
    "baseCommitSha": "4a9bc81f...",
    "files": [
      {
        "path": "src/auth/session.ts",
        "status": "modified",
        "additions": 8,
        "deletions": 2
      }
    ],
    "diff": "diff --git a/src/auth/session.ts ...",
    "validation": {
      "valid": true,
      "applied": true,
      "errors": []
    }
  }
}
```

Allowed file statuses: added, modified, deleted, renamed. Reject absolute paths, ../ traversal, symlink escapes, unexpected binary changes and changes outside the temporary workspace. Recheck base commit SHA before applying or publishing.

10.2 POST /agent/:runId/test

Starts or retries test execution for a validated patch. v1 initially supports Node.js repositories using predefined runner images and allowlisted commands. The backend must not execute arbitrary Dockerfiles or arbitrary model-provided shell commands.

Request

```
{
  "testProfile": "node-default",
  "commands": ["npm ci", "npm test"],
  "timeoutSeconds": 600
}
```

The server must map testProfile to a trusted runner definition. Commands must be checked against an allowlist/policy. A client must not be able to request privileged mode, host mounts, Docker socket access, unrestricted network or secret injection.

Response — 202 Accepted

```
{
  "success": true,
  "data": {
    "runId": "run_...",
    "testExecutionId": "test_...",
    "status": "queued"
  }
}
```

Test result object

```
{
  "status": "passed",
  "commands": [
    {
      "command": "npm test",
      "exitCode": 0,
      "stdout": "24 tests passed",
      "stderr": "",
      "durationMs": 1420,
      "timedOut": false
    }
  ],
  "startedAt": "2026-10-31T09:40:00Z",
  "completedAt": "2026-10-31T09:40:02Z",
  "sandbox": {
    "runner": "node-default",
    "cleanupCompleted": true
  }
}
```

Test statuses: queued, running, passed, failed, timed_out, cancelled, not_run. A failed test is a valid result and must not be hidden. Keep stdout/stderr size-limited and redact likely secrets before storing or returning logs.

11. Approval and pull-request publishing

11.1 POST /agent/:runId/approve

Purpose: record explicit user approval to publish the currently reviewed patch. Approval is scoped to the exact patch hash and base commit SHA.

Request

```
{
  "patchSha256": "sha256-of-reviewed-patch",
  "baseCommitSha": "4a9bc81f...",
  "approved": true
}
```

Response — 200 OK

```
{
  "success": true,
  "data": {
    "runId": "run_...",
    "approval": {
      "approved": true,
      "approvedAt": "2026-10-31T09:50:00Z",
      "patchSha256": "sha256-of-reviewed-patch"
    },
    "publishingEligible": true
  }
}
```

If the patch changes after approval, approval is invalidated and the user must review and approve the new patch. If tests have not run, the API may permit an explicit risk acknowledgement only if the product chooses to support that; the recommended v1 default is to require a completed test result or clearly mark the patch untested and block publishing.

11.2 POST /agent/:runId/pull-request

Creates a branch in the authenticated user's fork, commits the approved patch and opens a pull request against the original public repository. This operation requires explicit approval and an idempotency key.

Request

```
{
  "title": "Fix authentication refresh timeout",
  "body": "## Summary\n- Add retry handling...\n\n## Verification\n- npm test: passed",
  "targetBranch": "main"
}
```

Headers

```
Idempotency-Key: 9d61c2e5-6a4a-4d0c-a2a0-9a6c9f...
```

Response — 201 Created

```
{
  "success": true,
  "data": {
    "pullRequest": {
      "number": 129,
      "title": "Fix authentication refresh timeout",
      "htmlUrl": "https://github.com/owner/repository/pull/129",
      "headRepository": "contributor/repository",
      "headBranch": "repopilot/fix-issue-128",
      "baseRepository": "owner/repository",
      "baseBranch": "main",
      "state": "open",
      "createdAt": "2026-10-31T10:00:00Z"
    }
  }
}
```

Publishing safeguards

Before writing to GitHub: verify session and repository ownership; verify approval is current; verify patch hash and base commit; check the user's fork and write permissions; generate a collision-resistant branch; create the branch and commit in the fork; create the PR against upstream; store the returned PR URL. Never merge automatically. If any stage fails, report the exact completed actions and do not claim a PR exists.

12. Patch Evidence Report

12.1 GET /reports/:runId

Returns the report for an owned agent run. A report may be partial while a run is active; completed reports include all available evidence. A missing test result must be represented as not_run, not passed.


```
{
  "success": true,
  "data": {
    "reportId": "report_...",
    "runId": "run_...",
    "repository": {
      "fullName": "owner/repository",
      "htmlUrl": "https://github.com/owner/repository"
    },
    "issue": {
      "number": 128,
      "title": "Fix authentication timeout",
      "htmlUrl": "https://github.com/owner/repository/issues/128"
    },
    "investigation": {
      "summary": "A retry path may be skipped...",
      "possibleCauses": [],
      "uncertainties": []
    },
    "artifacts": {
      "inspectedFiles": ["src/auth/session.ts", "tests/auth.test.ts"],
      "changedFiles": ["src/auth/session.ts"],
      "patch": "diff --git ..."
    },
    "tests": {
      "status": "passed",
      "commands": [],
      "exitCode": 0,
      "durationMs": 1420
    },
    "pullRequest": {
      "url": "https://github.com/owner/repository/pull/129",
      "status": "created"
    },
    "createdAt": "2026-10-31T10:00:00Z"
  }
}
```


The report is an evidence record, not a guarantee that the change is correct. Preserve timestamps, commit SHAs, patch hash, commands and raw bounded outputs so the report can be audited.

13. Server-Sent Events contract

13.1 GET /agent/:runId/events

Streams timestamped execution events while a run is active. Use Accept: text/event-stream. The connection must be authenticated and authorized. Send Cache-Control: no-cache and Connection: keep-alive. Support Last-Event-ID for reconnect where event persistence is implemented.


```
event: stage.started
id: evt_0001
data: {"runId":"run_...","stage":"issue_analysis","timestamp":"2026-10-31T09:31:00Z"}

event: stage.completed
id: evt_0002
data: {"runId":"run_...","stage":"issue_analysis","status":"succeeded","durationMs":820}
event: agent.progress
```

```
id: evt_0003
data: {"runId":"run_...","currentStage":"repository_investigation","completedStages":1,
"totalStages":7}

event: agent.completed
id: evt_0010
data: {"runId":"run_...","status":"awaiting_review","reportAvailable":true}

event: agent.failed
id: evt_0011
data: {"runId":"run_...","stage":"patch_generation","code":"PATCH_GENERATION_FAILED","message":
"Could not produce a valid patch."}
```

13.2 Event types

Event	When emitted	Required payload fields
agent.queued	Run accepted into queue	runId, timestamp
agent.started	Orchestrator begins	runId, timestamp
stage.started	A stage begins	runId, stage, timestamp
stage.completed	A stage finishes successfully	runId, stage, status, durationMs
stage.failed	A stage fails	runId, stage, error code, timestamp
agent.progress	Progress snapshot changes	runId, currentStage, completedStages, totalStages
test.output	New bounded test output chunk	runId, executionId, stream, chunk
test.completed	Test execution ends	runId, executionId, status, exitCode, durationMs
agent.completed	Run reaches review/completion state	runId, status, reportAvailable
agent.failed	Run fails terminally	runId, stage, code, message

Send a comment/heartbeat event periodically to keep intermediaries from closing idle streams. On reconnect, replay events after Last-Event-ID if retained; otherwise instruct the client to fetch GET /agent/:runId for a fresh snapshot. SSE events are informational; the status endpoint remains the source of truth.

14. Data models and state machine

14.1 Core persistence collections

Collection	Important fields
users	_id, githubId, username, avatarUrl, encryptedCredentialReference, createdAt, updatedAt
repositories	_id, owner, name, fullName, githubRepositoryId, defaultBranch, primaryLanguage, description, readmeSummary, indexingStatus, connectedBy
agentRuns	_id, userId, repositoryId, issueNumber, status, currentStage, investigation, relevantFiles, generatedPatch, testResults, error, startedAt, completedAt
reports	_id, runId, repository, issue, investigationSummary, filesChanged, patch, testEvidence, pullRequestUrl, createdAt
approvals	_id, runId, userId, patchSha256, baseCommitSha, approvedAt, invalidatedAt
events	_id, runId, sequence, eventType, payload, timestamp

14.2 Agent run state machine

The exact terminal state policy should be implemented consistently. Recommended distinction: workflow execution ends at awaiting_review; after user action, the run becomes completed when finalized, or remains awaiting_review until the user approves or dismisses it. Publishing failures should preserve the evidence and allow a safe retry.

14.3 Data consistency rules

Store the repository default branch and base commit SHA used for analysis. A patch is valid only against the base SHA it was generated from. If upstream changes before publishing, compare the current base and either rebase/revalidate or reject as stale. Approval binds to the patch hash and base SHA. A PR URL is populated only after GitHub confirms creation.

15. Error catalogue

HTTP	Error code	Meaning / client action
400	VALIDATION_ERROR	Invalid request; show field-level details.
400	INVALID_REPOSITORY_URL	Provide a valid public GitHub repository URL.
401	UNAUTHENTICATED	Start or refresh GitHub login.
403	FORBIDDEN	User lacks permission for this resource/action.
403	GITHUB_WRITE_PERMISSION_REQUIRED	Explain fork/write permission requirement.
404	RESOURCE_NOT_FOUND	Resource absent or not owned by caller.
409	RUN_STATE_CONFLICT	Action not valid for current run state.
409	PATCH_STALE	Repository base changed; re-investigate/rebase.
409	APPROVAL_REQUIRED	Explicit approval is missing or invalidated.

HTTP	Error code	Meaning / client action
409	IDEMPOTENCY_CONFLICT	Same key was used with a different request body.
413	PAYLOAD_TOO_LARGE	Request or artifact exceeds configured limit.
422	PATCH_VALIDATION_FAILED	Patch format/path/apply validation failed.
429	RATE_LIMITED	Wait and retry according to Retry-After.
500	INTERNAL_ERROR	Unexpected backend error; include requestId in support report.
502	GITHUB_PROVIDER_ERROR	GitHub API failed; retry only safe operations.
502	MODEL_PROVIDER_ERROR	Gemma provider failed or returned invalid output.
503	SANDBOX_UNAVAILABLE	Docker worker is unavailable; retry later.
504	UPSTREAM_TIMEOUT	Provider or worker exceeded timeout.

Error responses must be stable and machine-readable. Do not expose provider response bodies directly. Include retryable=true only when a safe retry is appropriate.

16. Security and operational requirements

Area	Required controls
Authentication	GitHub OAuth state validation; secure session cookies; session expiry and logout.
Secrets	Encrypt credentials at rest; keep tokens server-side; redact logs; rotate secrets.
Authorization	Ownership checks on repository, run and report; least privilege.
Prompt injection	Treat issue text, README, comments and source code as untrusted data, never as system instructions.
Tool execution	Model cannot invoke shell or Docker directly; backend validates each tool request.
Patch safety	Reject path traversal, absolute paths, symlink escape and unsupported binary modifications.
Docker	Non-root user; no privileged mode; no Docker socket in runner; no host mounts; resource/time limits; restricted network; no production secrets; cleanup.
API abuse	Rate limits, request size limits, pagination caps and input validation.
Audit	Record who approved, patch hash, base SHA, PR action and timestamps.
Data handling	Bound logs and source excerpts; avoid storing unnecessary repository content.

16.1 Docker worker baseline

The worker must use a trusted predefined image for supported Node.js projects. Never run a repository-supplied Dockerfile as part of the default test path. Disable network by default after dependencies are available where feasible; if dependency installation requires network, use a constrained, explicit phase. Apply CPU, memory, process, disk and wall-clock limits. Destroy the container and temporary workspace after completion while retaining only required evidence.

17. Idempotency, pagination and rate limits

17.1 Idempotency

Require Idempotency-Key for POST /agent/:runId/pull-request. Strongly recommend it for POST /agent/analyze and POST /agent/:runId/test. Store the key, authenticated user, route, normalized request hash and result. Repeating the same key and same request returns the original result. Reusing a key with a different request returns 409 IDEMPOTENCY_CONFLICT.

17.2 Pagination

GitHub issue listing uses page/perPage. Cap perPage at 100 and prefer a smaller default. Return hasNextPage and optionally totalCount only when the upstream API can provide a reliable value. Do not fetch all repository issues by default.

17.3 Rate limiting

Apply per-user and per-IP limits. Suggested starting limits (tune after measurement): repository connect 10/minute; agent run creation 5/minute and 20/hour; test execution 5/minute; PR creation 3/minute; general read endpoints 60/minute. These are proposed operational defaults, not PRD-defined requirements. Return 429 with Retry-After.

18. Environment variables

Variable	Purpose	Secret?
NODE_ENV	Runtime environment	No
PORT	Express listen port	No
MONGODB_URI	MongoDB connection string	Yes
SESSION_SECRET	Application session signing secret	Yes
GITHUB_CLIENT_ID	GitHub OAuth app client id	Sensitive
GITHUB_CLIENT_SECRET	GitHub OAuth app secret	Yes
GITHUB_CALLBACK_URL	OAuth callback URL	No
GEMMA_API_KEY	Gemma inference provider key	Yes
GEMMA_API_BASE_URL	Gemma inference endpoint	No
DOCKER_WORKER_URL	Private worker service endpoint	Sensitive
DOCKER_WORKER_TOKEN	Worker authentication token	Yes
APP_BASE_URL	Frontend origin	No
API_BASE_URL	Backend public origin	No
LOG_LEVEL	Structured logging level	No
MAX_REPOSITORY_FILE_BYTES	Maximum source file size	No
MAX_CONTEXT_TOKENS	Model context budget	No
AGENT_STAGE_TIMEOUT_MS	Per-stage timeout	No
SANDBOX_TIMEOUT_SECONDS	Maximum test runtime	No

Never commit .env files. Provide .env.example with placeholders only. Production secrets should be managed by the hosting provider's secret store.

19. End-to-end integration example

The following sequence illustrates the expected client workflow. It is an integration example, not a claim that these endpoints have already been implemented.

```
# 1. Connect a public repository
POST /api/repositories/connect
{ "repositoryUrl": "https://github.com/owner/repository" }

# 2. List issues
GET /api/repositories/{repositoryId}/issues?page=1&perPage=20

# 3. Start analysis
POST /api/agent/analyze
{ "repositoryId": "{repositoryId}", "issueNumber": 128 }

# 4. Subscribe to events
GET /api/agent/{runId}/events
Accept: text/event-stream

# 5. Fetch current run snapshot
GET /api/agent/{runId}

# 6. Start tests after patch validation/review
POST /api/agent/{runId}/test
{ "testProfile": "node-default",
  "commands": ["npm ci", "npm test"],
  "timeoutSeconds": 600 }

# 7. Retrieve evidence
GET /api/reports/{runId}

# 8. Approve the exact patch
POST /api/agent/{runId}/approve
{ "patchSha256": "...", "baseCommitSha": "...", "approved": true }

# 9. Publish a PR (explicit user action)
POST /api/agent/{runId}/pull-request
Idempotency-Key: <unique-key>
{ "title": "Fix issue #128", "body": "..." }
```

19.1 Frontend state guidance

The frontend should treat API responses as the source of truth. Use the run snapshot to hydrate page state, then apply SSE events. On reconnect, fetch the latest snapshot. Disable approval until a patch is available and reviewed. Disable PR publishing until approval is valid. Display test status exactly as returned: passed, failed, timed_out or not_run.

20. Acceptance and test checklist

Area	Acceptance test
Authentication	OAuth state is validated; session works; GitHub token is never returned to browser.
Repository	Valid public repo connects; private/invalid repo is rejected clearly.
Issues	Real open issues load; pull requests are excluded; pagination works.
Classification	Gemma response is schema-validated; estimates are labelled.
Investigation	Evidence paths refer to retrieved files; uncertainty is preserved.
Patch	Valid patch applies to temporary workspace; invalid/traversal patch is rejected.
Sandbox	Actual test command runs in restricted container; exit code and duration captured.
Events	SSE reports real backend events; reconnect can recover via snapshot.
Evidence report	Inspected files, changed files, diff and test evidence are included.

Area	Acceptance test
Approval	Patch hash and base SHA are bound to explicit user approval.
PR publishing	PR created in user's fork only after approval; no automatic merge.
Errors	Stable error envelope and requestId returned; no secrets or stack traces leaked.

Appendix A — Implementation notes

Recommended backend modules: auth.routes/controller/service; repository.routes/controller/service; issue service; agent.routes/controller/orchestrator; model service; repository retrieval service; patch validation service; sandbox client/worker; report service; GitHub publishing service; SSE event publisher; shared schemas and error middleware.

Recommended validation: define Zod schemas for every body, query and path parameter. Keep provider-specific Gemma request/response formats behind a model adapter. Keep GitHub API operations behind a GitHub service. Keep Docker execution behind a worker interface so the API server does not need direct access to a privileged Docker daemon.

Appendix B — Contract decisions to confirm before implementation

Decision	Recommended v1 choice
Session transport	Secure HttpOnly cookie for browser app; bearer auth for non-browser clients if required.
OAuth callback	Backend callback route preferred; frontend code exchange only with strict state verification.
Long-running jobs	Background worker/queue if deployment supports it; persist run status in MongoDB.
Docker isolation	Separate restricted worker host if app platform cannot safely run Docker workloads.
Patch approval	Approval binds to patch SHA-256 and base commit SHA.
PR target	User's fork branch to original public repository default branch.
Supported test runtime	Node.js only for initial MVP.
Model output	Structured JSON validated server-side; no direct model-to-shell execution.

Definition of API v1 complete: A user can authenticate with GitHub, connect a public repository, select a real issue, start an agent run, observe real execution events, inspect a proposed patch, execute tests in a restricted sandbox, retrieve a Patch Evidence Report, approve the exact patch and create a pull request in their fork. No merge occurs automatically.